

2026년 오픈소스 개발자대회 결과보고서

항 목	내 용	항 목	내 용
팀 명	김대욱	팀 인 원 (팀장 포함)	1명
참가부문	일반	과제유형	지정과제(가이아쓰리디)

□ 결과보고서

프로젝트 개요	
프로젝트명	copc-cesium
프로젝트 등록 URL	https://github.com/thissak/copc-cesium
시연영상	https://youtu.be/g3pzx97skDU
프로젝트 소개	COPC(.copc.laz) 포인트클라우드 파일을 3D Tiles 로 사전 변환하지 않고 HTTP Range 요청으로 직접 읽어, CesiumJS 의 표준 Cesium3DTilesset 으로 노출하는 오픈소스 스트리밍 플러그인
프로젝트 세부 내용	
개발배경 및 목적	○ 문제 — 대용량 포인트클라우드를 웹 지도에 올리려면 원본을 3D Tiles 로 미리 변환(타일링)해야 한다. 기가바이트급은 변환에만 수십 분~수 시간이 걸리고, 원본과 변환본을 모두 보관해 용량이 두 배가 되며, 원본이 갱신될 때마다 변환 파이프라인을 다시 돌려야 한다. ○ 기회 — COPC 는 이미 옥트리를 내장한 클라우드 최적화 포맷이다. HTTP Range 로 필요한 노드만 부분 조회할 수 있으므로 원리적으로 사전 변환 단계는 불필요하다. ○ 공백 — 빌딩블록(copc.js-laz-perf-CesiumJS)은 공개돼 있으나 이를 잇는 재사용 가능한 오픈소스 통합 계층이 드물다. 상용 솔루션은 소스가 비공개이고, 오픈소스 뷰어는 대부분 CesiumJS 가 아닌 렌더러 기반이어서 "오픈소스 ∩ CesiumJS" 교집합이 비어 있었다. ○ 목표 — COPC URL 하나로 CesiumJS 에서 LOD 스트리밍이 동작하는 플러그인을, 사전 변환 없이, 표준 Cesium 타입을 반환하는 형태로 제공한다. (가이아쓰리디 지정과제 "COPC 데이터의 CesiumJS 가시화 기술 개

	발")
개발환경	○ 언어·빌드 — TypeScript 6.0(strict) · Vite 8(개발·데모) · tsup 8(배포 번들) ○ 런타임 — Node.js 24 / 최신 브라우저(WebGL2) ○ 렌더링 — CesiumJS 1.142 이상 (peer dependency) ○ 파싱·디코드 — copc.js 0.0.8 · laz-perf 0.0.7(WebAssembly) ○ 병렬·좌표 — Web Worker + Comlink 4.4 · proj4 2.20 ○ 검증 — Playwright 1.61 헤드리스 실브라우저 + 자체 측정 하네스 (4축 프로파일러) ○ 측정 환경 — Windows 11 / NVIDIA RTX 4090(ANGLE D3D11), macOS / Apple M4 Pro ○ 배포 — npm(@goldenlabs/copc-cesium) · Vercel(라이브 데모)
시스템 구성 및 아키텍처	○ 설계 원칙 — "LOD 를 새로 만들지 않고 CesiumJS 에 위임한다" ○ 데이터 흐름 — COPC 옥트리(원본 .copc.laz, HTTP Range) → 노드 1개 = 타일 1개로 매핑해 동적 3D Tiles 트리를 메모리에서 생성 → 어떤 노드를 언제 가져올지는 Cesium 이 판단(Screen Space Error) - 후 XHR 로 요청 → Service Worker 가 네트워크 계층에서 가로챈 → Web Worker 가 그 노드만 온디맨드 - 디코드(laz-perf WASM) → pnts 응답 → CesiumJS 렌더(LOD·컬링·GPU 메모리 관리 전부 Cesium 기본 동작) ○ 기술 선택의 근거 - Service Worker: Cesium 은 타일 content 를 fetch 가 아닌 XHR 로 가져온다는 것을 진단으로 확정. - fetch 패치로는 못 잡으므로 둘 다 잡는 네트워크 계층이 유일한 경로였다. - 표준 Cesium3DTileset 반환: wrapper 를 만들면 스타일·피킹·이벤트를 재구현해야 한다. - Web Worker 디코드: WASM 디코드가 메인 스레드를 막으면 렌더가 끊긴다. 4축 프로파일러로 병목을 측정해 디코드 축을 분리했다. ○ 설계 결정 근거는 저장소 docs/adr/ 에 ADR 7건으로 보존

프로젝트 주요기능	프로젝트 상세 내용 1. 한 줄 통합 — 사전 변환 0단계 <pre>const tileset = await CopcTileset.fromUrl('https://.../cloud.copc.laz');</pre> viewer.scene.primitives.add(tileset); 반환값이 표준 Cesium3DTileset 이라 기존 CesiumJS 코드에 그대로 엮힌다. 지정과제가 지목한 COG 의 TIFFImageryProvider.fromUrl(url) 과 동일한 1-라인 진입점 패턴이 다. 2. Cesium 스타일 언어 호환 LAS 분류·강도·리턴 수를 pnts batch table 로 노출해 Cesium3DTileStyle 선언적 스타일이 동작한다. 3. 원본 점 조회 (snapPoint) 화면 픽셀이 아니라 옥트리 최심 노드를 클릭 시점에 받아 디코드해 실제 최근접 원본 점의 좌표와 LAS 속성을 반환한다. 타일로 구운 데이터만으로는 구조적으로 제공 하기 어려운 기능이다. 4. 대용량 스트리밍 · 5가지 색상 모드 하이어라키 서브페이지를 온디맨드로 로드해 GB급 옥트리를 임의 깊이까지 스트리밍한다. RGB·고도· 분류·강도·리턴 수 중 선택하며, RGB 가 없으면 고도 색으로 자동 폴 백한다(백분위 클리핑 적용). 구동 및 시연 설치: npm install @goldenlabs/copc-cesium cesium → 서비스워커 를 사이트 루트에 복사 → fromUrl 호출 라이브 데모: https://copc-cesium.vercel.app 시연영상: https://youtu.be/g3pzx97skDU (2분 59초) ○ 자동 검증 18종 전부 통과 — npm test(오프라인 9/9: 좌표계 ·ECEF·피킹·pnts 배치·스타일·요청 스로틀·SW 라우팅·Cesium 코덱·공개 타입), npm run test:integration(실데이터 9/9: 실제 S3 COPC 로 전 파이프라인 검증) ○ 정확성 — ECEF 오차 최대 1.4×10^{-9} m · Autzen 중심 - 123.0688°, 44.0562°(소수점 4자리 일치) ○ 성능(자체 before/after) — S3 왕복 61→6회 · 재투영 582→10.7 ms/100만 점(54배) · 깊은 줌 16→89 fps · 8.9GB 90초 항해에서 메모리 plateau
----------------------	--

기대효과 및 활용분야	향후 확장성 및 기대효과 ○ 활용 분야 — 국토·도시(항공 LiDAR 수치표고모형, 도시 3D 모델링, 건축물 변화 탐지) · 재난·안전(산사태·침수 지형 분석, 재난 전후 비교) · 인프라(도로·교량·송전선로 MLS 점검) · 문화재(정밀 스캔 데이터의 웹 공개) ○ 확장성 — 사전 변환 파이프라인이 없으므로 측량 데이터를 오브젝트 스토리지에 올리는 즉시 웹에서 볼 수 있다. 갱신 주기가 짧은 모니터링 용도에서 효과가 특히 크며, 반환값이 표준 Cesium3DTileset 이라 기존 CesiumJS 기반 GIS 제품에 별도 통합 계층 없이 얹힌다. ○ 기대효과 — 변환 서버·스토리지 이중화 비용 제거 · Apache-2.0 으로 상용 통합 제약 없음 · npm 배포로 도입 비용 최소화
기타	프로젝트의 혁신성 및 차별성 ■ 사전 변환 단계를 없앴다 COPC 가 이미 갖고 있는 옥트리를 그대로 3D Tiles 트리로 노출해 타일링 단계를 제거했다. 변환 시간 0 · 변환 서버 0 · 저장 용량 1배(원본만) · 원본 갱신 시 재변환 없이 즉시 반영. ■ 표준 Cesium3DTileset 을 반환한다 전용 wrapper 가 아니라 Cesium 표준 타입을 그대로 돌려주므로 Cesium3DTileStyle 선언적 스타일, 피킹·tileLoad 이벤트, LOD·절두체 컬링·GPU 메모리 관리가 추가 코드 없이 동작한다. TypeScript strict 에서 Cesium3DTileset 변수에 직접 할당해도 타입 오류가 없음을 배포본으로 확인했다. ■ 원본 점에 직접 접근한다 (snapPoint) 화면 픽셀이 아니라 옥트리 최심 노드를 클릭 시점에 받아 디코드해 실제 최근접 원본 점의 좌표와 LAS 속성(분류·강도·리턴)을 반환한다. 타일로 구운 데이터에는 원본 점이 남지 않아 구조적으로 하기 어려운 일이며, 측량 데이터를 "보는 것"에서 "질의하는 것"으로 확장한다. ■ 추측이 아니라 측정이 설계를 결정했다 4축 프로파일러(요청·디코드·좌표변환·렌더)를 먼저 만들고 병목을 축 단위로 분리 측정했다. S3 range 왕복 61→6회 · 재투영 582→10.7 ms/100만 점(54배, 비중 50%→2%) · 깊은 줌 16→89 fps · 위치 바이트 절반(POSITION_QUANTIZED) · geometricError 식의 under-refine 을 측정으로 확인해 교체.

반증된 기능은 넣지 않았다 — CustomShader 는 셰이더 컴파일 실패로 렌더가 멈추는 것을 확인했다.

검증도 같은 원칙이다. 자동 검증 18종을 상시 회귀 가드로 두고 좌표 정합을 소수점 4자리로 확인했으며

(Autzen -123.0688°, 44.0562°), ECEF 오차 최대 1.4×10^{-9} m, 8.9GB 90초 항해에서 메모리 plateau 다.

■ 쓸 수 있는 상태로 완결했다

npm 배포(@goldenlabs/copc-cesium)와 라이브 데모까지 마쳐 설치·실행을 직접 확인할 수 있고,

Apache-2.0 이라 상용 통합에 제약이 없다.

한계점 및 향후 발전 로드맵

○ 한계 — (1) Service Worker 필수: 스코프가 파일 위치로 결정되어 소비자 origin 에 워커 파일을 복사

하는 단계가 필요하다(msw 등과 동일한 관행). 가로채기 불가 시 명확한 오류를 던진다.

(2) CustomShader 미지원(위 사유). (3) 색 범위 클리핑 백분위가 고정값.

○ 로드맵 — 단기: 노브 옵션화·예제 확충 / 중기: 3D Tiles 1.1-gltf content 포맷 검토 /

장기: 시간축(4D) 스트리밍, 속성 기반 실시간 필터링 / 유지보수: 검증 18종 CI 연결

소감 및 후기

가장 크게 배운 것은 추측을 측정으로 바꾸는 습관이었다. 병목을 디코드 속도로 짐작했으나 4축

프로파일러로 재보니 실제 지배 요인은 S3 range 왕복 횟수였고, 요청을 병합하자 61회가 6회로 줄었다.

반대로 측정 때문에 포기한 기능도 있다. 제출 직전 시연영상을 다른 관점에서 교차검토받는 과정에서는

데모의 노드 카운터가 갱신되지 않는 결함과 클릭 후 수 초간 반응이 없는 UX 결함을 발견해 근본

원인까지 고쳤다. 영상을 만들려다 제품이 좋아진 셈이다. 혼자 하는 프로젝트일수록 자기 결과물을

낯선 눈으로 다시 보는 절차가 필요하다는 것을 배웠다.

붙임1

SBOM[소프트웨어 자재명세서]

번호	라이브러리명	버전	라이선스	공식 저장소 URL(GitHub 등)	사용 목적 및 주요 기능
1	cesium	1.142.0	Apache-2.0	https://github.com/CesiumGS/cesium	3D 지구본 렌더링 엔진. peer dependency 로 참조하며 LOD·컬링·GPU 메모리 관리를 위임
2	copc	0.0.8	MIT	https://github.com/connormanning/copc.js	COPC 파일 파싱. 헤더·VLR·옥트리 하이어나 키 읽기 및 HTTP Range getter 제공
3	laz-perf	0.0.7	Apache-2.0	https://github.com/hobuinc/laz-perf	LAZ 압축 포인트 데이터 디코딩 (WebAssembly). Web Worker 안에서 노드 단위 디코드
4	comlink	4.4.2	Apache-2.0	https://github.com/GoogleChromeLabs/comlink	Web Worker RPC 통신. 디코드 작업을 메인 스레드 밖으로 위임
5	proj4	2.20.9	MIT	https://github.com/proj4js/proj4js	좌표계 변환. LAS 헤더의 WKT CRS 를 WGS84 로 재투영
6	p-retry	8.0.0	MIT	https://github.com/sindresorhus/p-retry	HTTP Range 요청 실패 시 재시도 및 타임아웃 처리
7	vite	8.0.16	MIT	https://github.com/vitejs/vite	개발 서버 및 데모 번들링 (개발 의존성)
8	vite-plugin-cesium	1.2.23	MIT	https://github.com/nshen/vite-plugin-cesium	Cesium 정적 에셋 처리 (개발 의존성)
9	tsup	8.5.1	MIT	https://github.com/egoist/tsup	라이브러리 배포 번들 생성 (개발 의존성)
10	typescript	6.0.3	Apache-2.0	https://github.com/microsoft/TypeScript	타입 검사 및 컴파일 (개발 의존성)

번호	라이브러리명	버전	라이선스	공식 저장소 URL(GitHub 등)	사용 목적 및 주요 기능
11	tsx	4.22.4	MIT	https://github.com/privatenumber/tsx	검증 스크립트 실행기 (개발 의존성)
12	playwright	1.61.0	Apache-2.0	https://github.com/microsoft/playwright	헤드리스 실브라우저 자동 검증 (개발 의존성)
13	@types/proj4	2.5.6	MIT	https://github.com/DefinitelyTyped/DefinitelyTyped	proj4 타입 정의 (개발 의존성)

※ 필요 시, 행을 추가하여 기재해 주시기 바랍니다.

붙임2

AI 모델 활용 및 라이선스 기술 명세서

[작성 안내]

- 본 서식은 프로젝트에 탑재·적용된 AI 모델의 오픈소스 요건 및 라이선스 준수 여부를 검증하기 위한 기술 명세서입니다.
- 참가팀은 본인의 AI 개발 유형(유형1, 2, 3)을 먼저 선택한 후 해당하는 항목의 명세를 정확히 작성하여 제출해 주시기 바랍니다.

1. AI 모델 활용 유형 (해당하는 항목에 ☒ 표시)

☐ 유형 1: 외부 모델 그대로 활용 ☐ 유형 2: 외부 모델 파인튜닝 ☐ 유형 3: 자체 개발 모델

☒ 해당 없음 — 본 프로젝트는 AI 모델을 탑재·적용하지 않는다.

개발 과정에서 코딩 및 디버깅 보조용으로 상용 AI 를 활용했으며, 양식 안내에 따라
해당 내용은 4번 항목에 기재한다.

2. 기반(베이스) 모델 정보 (유형 1, 유형 2 작성 필수 / 유형 3은 '해당 없음'으로 기재)

기반 모델명 및 개발사	해당 없음 (AI 모델 미탑재)	기반 모델 라이선스	해당 없음
--------------	-------------------	------------	-------

3. 데이터셋 정보 및 가중치 배포 명세 (유형 2, 유형 3 작성 필수)

학습 데이터셋 정보 (출처 및 규모)	해당 없음 (학습 미수행)
데이터 정제/ 가공 방법 요약	해당 없음
새로 생성된 가중치 공개 저장소 URL	해당 없음

가중치 파일 정보 및 배포방식	해당 없음		
4. 소스코드 라이선스 및 개발 환경 정보 (모든 유형 필수 작성)			
직접 작성한 코드의 오픈소스 라이선스	Apache License 2.0	학습/추론 소스코드 공개 저장소 URL	https://github.com/thissak/copc-cesium
상용 AI 보조도구 활용 여부 및 범위	코드 작성 및 디버깅 보조용으로 Anthropic Claude 를 활용했다. AI 가 생성한 코드는 전 항목을 사람이 검토했고, 자동 검증 18종(오프라인 9 + 실데이터 9)과 헤드리스 실브라우저 테스트로 동작을 확인했다. 성능·정확성 주장은 재현 가능한 측정 하네스로 뒷받침하며, 측정으로 반증된 기능은 채택하지 않았다.		